STM – DOCKER E KUBERNETES

Instrutoria Interna

Docker Compose e Aplicações Multicontainer

Orquestração declarativa de ambientes de desenvolvimento e infraestrutura

Wescley de Macêdo Sousa
wescleysousa@stm.jus.br

DOCKER COMPOSE E APLICAÇÕES MULTICONTAINER

SUMÁRIO

MÓDULO 1: O IMPERATIVO DO DOCKER COMPOSE **3**
1.1 A PROBLEMÁTICA DO GERENCIAMENTO MANUAL VIA CLI (DOCKER RUN) 3
1.2 INTRODUÇÃO AO DOCKER COMPOSE E INFRAESTRUTURA COMO CÓDIGO (IAC) 5
MÓDULO 2: ANATOMIA E SINTAXE DECLARATIVA DO DOCKER-COMPOSE.YAML **7**
2.1 A ESPECIFICAÇÃO COMPOSE (COMPOSE SPECIFICATION) 7
2.2 ESTRUTURA DO ARQUIVO DOCKER-COMPOSE.YAML 7
2.3 O BLOCO SERVICES 8
2.4 O BLOCO NETWORKS 10
2.5 O BLOCO VOLUMES 11
2.6 GESTÃO ESTRUTURADA DE VARIÁVEIS DE AMBIENTE 11
2.7 DEPENDÊNCIAS AVANÇADAS E CHECAGENS DE SAÚDE (DEPENDS_ON + HEALTHCHECK) 14
MÓDULO 3: OPERAÇÃO, GERENCIAMENTO E CICLO DE VIDA DO AMBIENTE **15**
3.1 COMANDOS DE INICIALIZAÇÃO, INTERRUPÇÃO E DESCARTE 15
3.2 OBSERVABILIDADE, AUDITORIA E TROUBLESHOOTING 17
MÓDULO 4: BOAS PRÁTICAS DE ARQUITETURA E PRODUÇÃO **19**
4.1 O PADRÃO DE SOBRESCRITA COM DOCKER-COMPOSE.OVERRIDE.YAML 19
4.2 GERENCIAMENTO DE SERVIÇOS OPCIONAIS VIA PROFILES 21
MÓDULO 5: LABORATÓRIO PRÁTICO GUIADO (DESAFIO MULTICONTAINER) **23**
GUIA RÁPIDO DE COMANDOS DOCKER COMPOSE **34**
REFERÊNCIAS **35**

MÓDULO 1: O IMPERATIVO DO DOCKER COMPOSE

1.1 A PROBLEMÁTICA DO GERENCIAMENTO MANUAL VIA CLI (DOCKER RUN)

À medida que uma arquitetura de software evolui de um monolito simples para uma estrutura baseada em microsserviços, a complexidade operacional de gerenciamento de containers via linha de comando (`Docker CLI`) cresce exponencialmente.

Em um ambiente de desenvolvimento ou produção real, uma aplicação moderna raramente funciona isolada. Ela necessita de uma camada de apresentação (Frontend Web), um serviço de negócios (API/Backend), um banco de dados relacional (ex: PostgreSQL), um cache em memória (ex: Redis) e um mensageiro (ex: RabbitMQ).

Os Gargalos do Fluxo Manual:

1. **Extensão e Fragilidade dos Comandos CLI:** Para subir o ambiente manualmente, o desenvolvedor precisa executar uma sequência extensa de comandos no terminal, garantindo a ordem exata das dependências:

Bash

```bash
# 1. Criar a rede compartilhada
docker network create rede-aplicacao

# 2. Criar o volume para persistência do banco
docker volume create dados-postgres

# 3. Subir o Banco de Dados
docker run -d \
  --name db-postgres \
  --network rede-aplicacao \
  --mount source=dados-postgres,target=/var/lib/postgresql/data \
  -e POSTGRES_USER=app_user \
  -e POSTGRES_PASSWORD=senha_secreta \
  -e POSTGRES_DB=empresa_db \
  postgres:15-alpine

# 4. Compilar e subir a API Backend
docker build -t api-backend:v1 ./backend
docker run -d \
  --name api-backend \
  --network rede-aplicacao \
  -p 5000:5000 \
  -e DB_HOST=db-postgres \
  -e DB_USER=app_user \
  -e DB_PASSWORD=senha_secreta \
  -e DB_NAME=empresa_db \
  api-backend:v1

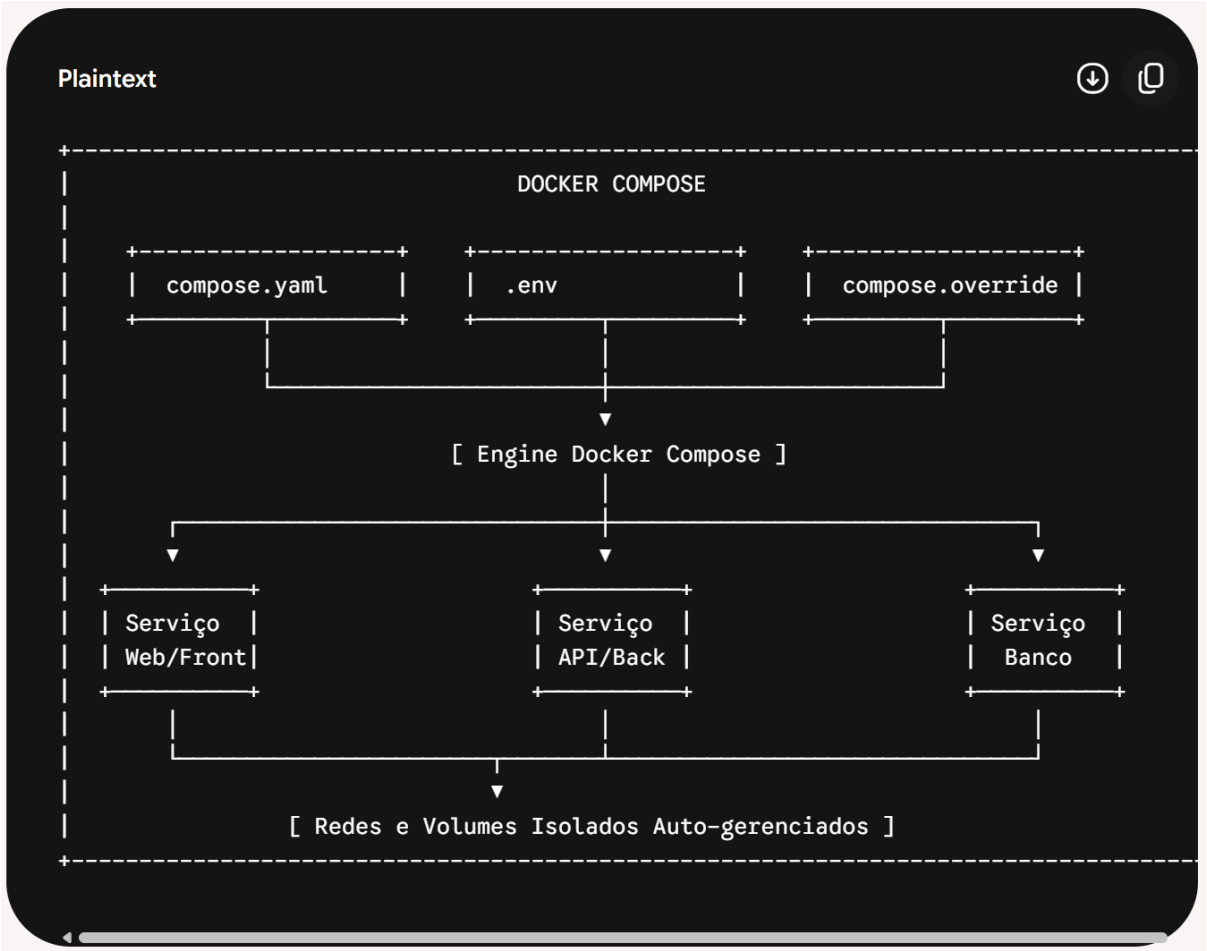
# 5. Compilar e subir o Frontend Web
docker build -t web-frontend:v1 ./frontend
docker run -d \
  --name web-frontend \
  --network rede-aplicacao \
  -p 80:80 \
  -e API_URL=http://api-backend:5000 \
  web-frontend:v1
```

1. **Propensão a Erros Humanos:** A digitação manual de parâmetros como portas (`-p`), variáveis de ambiente (`-e`), pontos de montagem (`--mount`) e redes (`--network`) introduz um alto risco de discrepâncias entre ambientes de diferentes desenvolvedores.
2. **Ausência de Rastreabilidade e Versionamento:** Os comandos executados no terminal não ficam salvos no repositório de código de forma declarativa. Scripts em Shell (`.sh`) costumam ser usados como paliativos, mas carecem de controle de estado, tratamento de falhas e padronização entre sistemas operacionais (Linux, macOS, Windows).
3. **Complexidade no Cleanup (Encerramento do Ambiente):** Destruir o ambiente exige executar `docker stop` e `docker rm` individualmente para cada container, além de remover redes e volumes manualmente.

1.2 INTRODUÇÃO AO DOCKER COMPOSE E INFRAESTRUTURA COMO CÓDIGO (IAC)

O **Docker Compose** é uma ferramenta oficial da Docker desenvolvida para definir, orquestrar e executar aplicações multi-container. Ele adota o paradigma de **Infraestrutura como Código (Infrastructure as Code - IaC)**.

Em vez de executar múltiplos comandos imperativos na linha de comando, toda a arquitetura da aplicação — serviços, redes, volumes, variáveis e ordens de inicialização — é declarada em um único arquivo de configuração no formato YAML (geralmente denominado docker-`compose.yaml`).



Principais Vantagens do Docker Compose:

- **Ponto Único de Configuração:** Todo o ecossistema da aplicação é documentado e versionado junto ao código-fonte no Git.
- **Comando Único de Controle:** É possível subir toda a pilha de serviços com `docker compose up -d` e destruí-la com `docker compose down`.
- **Isolamento de Ambientes por Projeto:** Por padrão, o Docker Compose isola os ambientes criando uma rede dedicada e prefixando os nomes dos recursos com o nome do diretório do projeto.
- **Gerenciamento Inteligente de Ciclo de Vida:** O Compose recria apenas os containers cujas configurações ou imagens foram alteradas, preservando o estado dos demais.

MÓDULO 2: ANATOMIA E SINTAXE DECLARATIVA DO DOCKER-COMPOSE.YAML

2.1 A ESPECIFICAÇÃO COMPOSE (COMPOSE SPECIFICATION)

A comunidade Docker padronizou a especificação do arquivo de orquestração sob a **Compose Specification**.

> 💡 **Nota Histórica e de Sintaxe:** Antigamente, utilizava-se a ferramenta legada em Python `docker-compose` (com hífen) e arquivos nomeados obrigatoriamente como `docker-compose.yml`, contendo um campo inicial `version: '3.8'`. Atualmente, na especificação moderna integrada diretamente à CLI do Docker como plugin (`docker compose`, sem hífen), a diretiva `version` tornou-se **obsoleta e opcional**. O nome de arquivo padrão preferencial adotado é **`compose.yaml`** (embora `docker-compose.yaml` continue sendo suportado por retrocompatibilidade).

2.2 ESTRUTURA DO ARQUIVO DOCKER-COMPOSE.YAML

O arquivo `docker-compose.yaml` possui uma estrutura hierárquica baseada em quatro blocos de primeiro nível:

```yaml
services:
  # Definição dos containers que compõem a aplicação

networks:
  # Definição das redes virtuais customizadas

volumes:
  # Definição dos volumes gerenciados para persistência

configs / secrets:
  # Gestão de arquivos de configuração e dados sensíveis
```

2.3 O BLOCO SERVICES

O bloco `services` é o coração do `compose.yaml`. Cada chave abaixo de `services` representa um container que será gerenciado pelo Compose.

Atributos Fundamentais de um Serviço:

1. `image` vs `build`

Define a origem da imagem do container.

- `image`: Baixa uma imagem pronta de um registro (Docker Hub, GHCR, ECR).
- `build`: Compila uma imagem localmente a partir de um `Dockerfile`.

YAML

```yaml
services:
  # Exemplo 1: Utilizando imagem pronta
  cache:
    image: redis:7-alpine

  # Exemplo 2: Compilando imagem local
  api:
    build:
      context: ./backend        # Diretório do código-fonte
      dockerfile: Dockerfile    # Nome do Dockerfile (opcional se for Dockerfile)
      args:                     # Argumentos de compilação (ARG)
        NODE_ENV: development
```

2. `ports` vs `expose`

- `ports`: Mapeia e publica portas do container no host hospedeiro (`HOST:CONTAINER`). Permite que o tráfego externo acesse o serviço.
- `expose`: Expõe portas **apenas para outros serviços na mesma rede do Docker**, sem abri-las para a máquina host externa.

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "80:80"        # Host 80 -> Container 80

  banco:
    image: postgres:15-alpine
    expose:
      - "5432"         # Acessível APENAS por outros containers na mesma rede
```

3. `restart` (Política de Reinicialização)

Define o comportamento do Docker Daemon caso o container caia ou falhe:

- `no`: Não reinicia o container automaticamente (padrão).
- `always`: Reinicia sempre o container se ele parar, independentemente do código de saída.
- `on-failure`: Reinicia apenas se o container falhar com um código de erro diferente de zero (`exit code != 0`).
- `unless-stopped`: Reinicia o container sempre, a menos que ele tenha sido explicitamente parado pelo usuário.

```yaml
services:
  servico-critico:
    image: my-app:v1
    restart: unless-stopped
```

2.4 O BLOCO NETWORKS

Por padrão, quando o Docker Compose é executado, ele cria automaticamente uma rede do tipo `bridge` nomeada no formato `<nome_do_diretorio>_default` e conecta todos os serviços a ela.

No entanto, para arquiteturas seguras de produção, é recomendada a criação de **Redes Customizadas** com controle de isolamento (*segmentação de rede*).

YAML

```yaml
services:
  frontend:
    image: nginx:alpine
    networks:
      - rede-publica

  backend:
    build: ./backend
    networks:
      - rede-publica
      - rede-privada

  database:
    image: postgres:15-alpine
    networks:
      - rede-privada    # O banco NÃO tem acesso à rede-publica nem à internet diret

networks:
  rede-publica:
    driver: bridge
  rede-privada:
    driver: bridge
    internal: true      # Impede completamente o tráfego de saída para a internet
```

2.5 O BLOCO VOLUMES

Permite declarar **Volumes Gerenciados** que sobrevivem à destruição dos containers com o comando `docker compose down`.

YAML

```yaml
services:
  db:
    image: postgres:15-alpine
    volumes:
      # Volume Gerenciado (Declarado na raiz do compose.yaml)
      - dados-pg:/var/lib/postgresql/data
      # Bind Mount (Mapeamento direto de pasta/arquivo do host)
      - ./config/postgres.conf:/etc/postgresql/postgres.conf:ro

volumes:
  dados-pg:
    driver: local      # Driver padrão de armazenamento no host
```

> 💡 **Flag `:ro` (Read-Only):** *Ao adicionar `:ro` ao final de um `Bind Mount`, o container ganha permissão apenas de* **leitura** *sobre os arquivos do host, impedindo que o container altere arquivos de configuração locais por acidente.*

2.6 GESTÃO ESTRUTURADA DE VARIÁVEIS DE AMBIENTE

O Docker Compose oferece três mecanismos complementares para a injeção de variáveis de ambiente em containers.

1. Atributo `environment` (Injeção Direta no YAML)

As variáveis são declaradas diretamente no arquivo docker-`compose.yaml`.

```yaml
services:
  api:
    image: node-api:v1
    environment:
      - PORT=3000
      - NODE_ENV=production
      - DB_HOST=db
```

2. Atributo `env_file` (Injeção via Arquivos `.env` de Runtime)

Carrega um ou múltiplos arquivos contendo pares `CHAVE=VALOR` para dentro do container em tempo de execução.

```yaml
services:
  api:
    image: node-api:v1
    env_file:
      - ./config/comum.env
      - ./config/segredo.env
```

3. Interpolação de Variáveis via Arquivo `.env` do Compose

O Docker Compose lê automaticamente um arquivo chamado `.env` localizado na **mesma raiz** do arquivo docker-`compose.yaml` e substitui os valores dinamicamente no próprio código YAML:

Arquivo `.env` (no host):

```
Plaintext

PROJECT_PORT=8080
TAG_VERSAO=v2.1.0
DB_PASS=senha_super_segura
```

Arquivo `docker-compose.yaml`:

```yaml
YAML

services:
  web:
    image: minha-app:${TAG_VERSAO}
    ports:
      - "${PROJECT_PORT}:80"
    environment:
      - DB_PASSWORD=${DB_PASS}
```

Tabela de Precedência de Variáveis de Ambiente:

Quando uma mesma variável é definida em múltiplos locais, o Docker Compose aplica a seguinte hierarquia de sobrescrita (do **mais forte** para o **mais fraco**):

1. Variáveis setadas na CLI do Docker Compose ou ambiente do Shell executante.
2. Variáveis definidas no bloco `environment` do `docker-compose.yaml`.
3. Variáveis vindas de arquivos declarados via `env_file`.
4. Variáveis declaradas no `Dockerfile` (ENV).

2.7 DEPENDÊNCIAS AVANÇADAS E CHECAGENS DE SAÚDE (DEPENDS_ON + HEALTHCHECK)

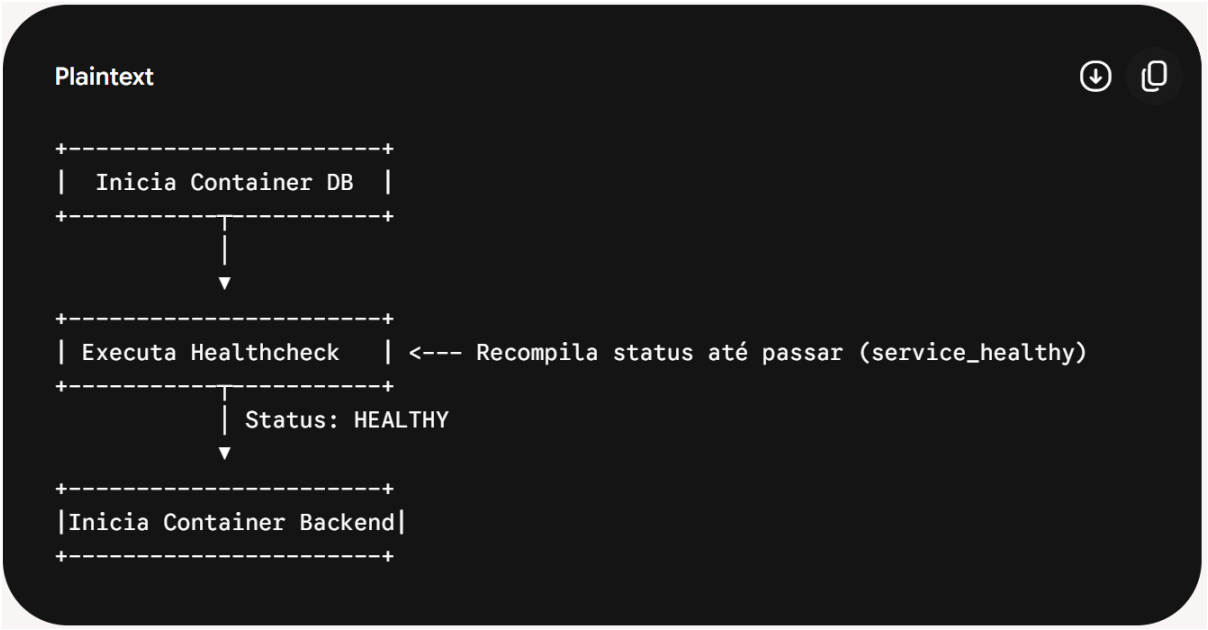
Por padrão, a instrução `depends_on` garante apenas a **ordem de inicialização** dos containers. Isso significa que o Compose iniciará o container `B` após mandar criar o container `A`. No entanto, isso **não garante** que o serviço dentro do container `A` já esteja pronto e aceitando conexões (por exemplo, um banco de dados PostgreSQL pode levar 10 segundos para inicializar suas tabelas).

Para resolver essa condição de corrida (*Race Condition*), combina-se `depends_on` com a instrução `healthcheck`.

YAML

```yaml
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: secretpassword
      POSTGRES_USER: admin
      POSTGRES_DB: appdb
    healthcheck:
      # Testa a conexão diretamente no banco via pg_isready
      test: ["CMD-SHELL", "pg_isready -U admin -d appdb"]
      interval: 5s       # Intervalo entre os testes
      timeout: 5s        # Tempo limite de resposta de cada teste
      retries: 5         # Número de falhas consecutivas para considerar 'unhealthy'
      start_period: 10s  # Tempo de carência inicial antes de começar a testar

  backend:
    build: ./backend
    ports:
      - "3000:3000"
    depends_on:
      db:
        condition: service_healthy    # O backend SÓ INICIA se o db estiver HEALTHY!
```



MÓDULO 3: OPERAÇÃO, GERENCIAMENTO E CICLO DE VIDA DO AMBIENTE

3.1 COMANDOS DE INICIALIZAÇÃO, INTERRUPÇÃO E DESCARTE

A CLI do Docker Compose (`docker compose`) padroniza o ciclo de vida da aplicação em comandos simples executados no mesmo diretório do arquivo `docker-compose.yaml`.

Sintaxe e Comandos Fundamentais:

1. `docker compose up`

Lê o `compose.yaml`, cria redes, volumes e inicia todos os containers necessários.

- `-d` (*Detached Mode*): Executa os containers em segundo plano, liberando o terminal.
- `--build`: Força a recompilação das imagens cujos diretórios de `build` sofreram alterações no código-fonte antes de subir os serviços.

```bash
# Sobe todo o ambiente compilando alterações e rodando em background
docker compose up -d --build
```

2. `docker compose down`

Interrompe e **remove** os containers, redes padrão e links criados pelo `up`.

- `-v` ou `--volumes`: Remove também os **Volumes Gerenciados** declarados no bloco `volumes`. **Atenção:** Isso apaga os dados persistidos do banco de dados!
- `--rmi all`: Remove todas as imagens utilizadas pelos serviços.

```bash
# Para e remove os containers e redes
docker compose down

# Para, remove containers, redes E apaga os volumes de dados
docker compose down -v
```

3. Interrupção e Reinicialização Sem Descarte (`stop`, `start`, `restart`)

- `docker compose stop`: Interrompe os containers sem removê-los do disco.
- `docker compose start`: Reinicia containers que estavam pausados/parados.
- `docker compose restart`: Reinicia o processo dentro dos containers ativos.

```bash
# Reinicia apenas o serviço de backend sem afetar o banco de dados
docker compose restart backend
```

3.2 OBSERVABILIDADE, AUDITORIA E TROUBLESHOOTING

Quando um ecossistema multi-container apresenta comportamentos inesperados, o Docker Compose oferece ferramentas avançadas para isolamento e diagnóstico de falhas.

Comandos de Inspeção:

1. `docker compose ps`

Exibe o estado atual de todos os serviços vinculados ao projeto atual, mapeamento de portas e status de saúde (*health status*).

```bash
docker compose ps
```

2. `docker compose logs`

Centraliza a leitura e agregação das saídas padrão (`stdout` e `stderr`) de **todos** os serviços do ambiente em uma única linha do tempo visual.

- `-f` (*Follow*): Acompanha os logs em tempo real.
- `--tail=N`: Exibe apenas as últimas N linhas do log.
- `<nome_do_servico>`: Filtra os logs de um serviço específico.

```bash
# Acompanha em tempo real os logs das últimas 50 linhas do serviço 'backend'
docker compose logs -f --tail=50 backend
```

3. `docker compose exec`

Executa um comando ou abre uma sessão interativa de terminal dentro de um container em execução.

```bash
# Abre um Shell interativo dentro do container do serviço 'db'
docker compose exec -it db psql -U admin -d appdb
```

4. `docker compose top`

Exibe os processos ativos em execução no nível do Sistema Operacional hospedeiro para cada um dos serviços.

```bash
docker compose top
```

5. `docker compose config`

Valida e exibe a estrutura final mesclada do arquivo docker-`compose.yaml` após resolver todas as variáveis de ambiente e arquivos de extensão (*override*). Útil para detectar erros de sintaxe YAML (como indentação incorreta) antes da execução.



MÓDULO 4: BOAS PRÁTICAS DE ARQUITETURA E PRODUÇÃO

4.1 O PADRÃO DE SOBRESCRITA COM DOCKER-COMPOSE.OVERRIDE.YAML

Em fluxos de desenvolvimento corporativo, as configurações necessárias para rodar um projeto na máquina local do desenvolvedor são muito diferentes das configurações de homologação e produção.

O Docker Compose possui o mecanismo nativo de **Múltiplos Arquivos de Configuração**. Por padrão, ele mescla dois arquivos automaticamente quando presentes:

1. `compose.yaml` (Contém as definições genéricas do projeto).
2. `compose.override.yaml` (Contém configurações exclusivas de desenvolvimento local, como montagem de código em tempo real e portas expostas).

Exemplo Prático de Divisão de Ambientes:

Base: `docker-compose.yaml` (Comum a todos os ambientes)

```yaml
services:
  api:
    build: ./backend
    environment:
      NODE_ENV: production
```

Desenvolvimento: `docker-compose.override.yaml` (Aplicado automaticamente em dev)

```yaml
services:
  api:
    environment:
      NODE_ENV: development
    volumes:
      # Mapeia o código local do host para live-reload dentro do container
      - ./backend:/app
    ports:
      - "3000:3000"
```

Produção: `docker-compose.prod.yaml` (Invocado explicitamente em pipelines CI/CD)

```yaml
services:
  api:
    restart: always
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: 512M
```

Para subir a aplicação em ambiente de Produção explicitando os arquivos desejados:

```bash
docker compose -f compose.yaml -f compose.prod.yaml up -d
```

4.2 GERENCIAMENTO DE SERVIÇOS OPCIONAIS VIA PROFILES

O recurso de **Perfis (`profiles`)** permite declarar serviços secundários no arquivo docker-`compose.yaml` (como ferramentas de depuração, testes integrados, administração de banco de dados) que **não sobem por padrão**, a menos que o perfil seja ativado explicitamente.

YAML

```yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "80:80"

  db:
    image: postgres:15-alpine

  # Ferramenta gráfica de administração de banco (Opcional)
  pgadmin:
    image: dpage/pgadmin4
    ports:
      - "5050:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin
    profiles:
      - tools      # Atribuído ao perfil 'tools'
```

Como Executar:

Bash

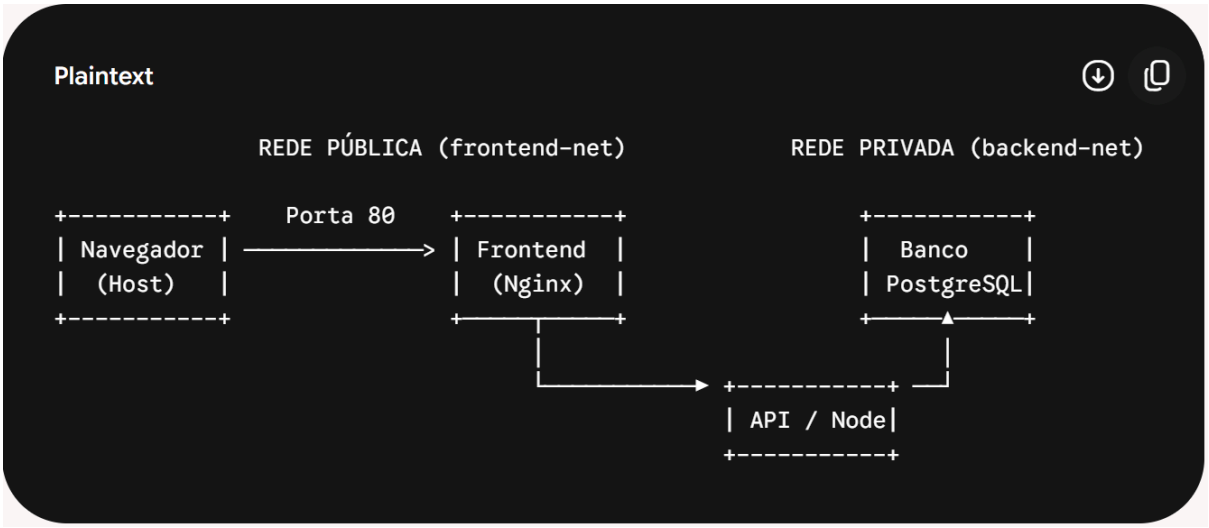
```bash
# Sobe APENAS 'web' e 'db' (padrão)
docker compose up -d

# Sobe 'web', 'db' E ativa o serviço do perfil 'tools' (pgadmin)
docker compose --profile tools up -d
```

MÓDULO 5: LABORATÓRIO PRÁTICO GUIADO (DESAFIO MULTICONTAINER)

Neste laboratório, construiremos uma arquitetura corporativa completa dividida em 3 camadas (*3-Tier Architecture*):

1. **Frontend Web:** Servidor Nginx servindo uma página estática e atuando como Proxy.
2. **Backend/API:** Aplicação Node.js/Express.
3. **Banco de Dados:** PostgreSQL com volume de dados persistente.



Passo 1: Estrutura de Diretórios do Projeto

Crie a seguinte estrutura de pastas e arquivos no seu terminal:

```
Plaintext

meu-projeto-multicontainer/
├── .env
├── compose.yaml
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
└── frontend/
    ├── Dockerfile
    ├── index.html
    └── nginx.conf
```

Passo 2: Código-Fonte da Camada Backend (API)

1. **backend/package.json:**

```
JSON

{
  "name": "api-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.3"
  }
}
```

1. `backend/server.js`:

```javascript
const express = require('express');
const { Pool } = require('pg');

const app = express();
app.use(express.json());

const pool = new Pool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  port: 5432,
});

// Endpoint de verificação de saúde da API
app.get('/api/health', async (req, res) => {
  try {
    await pool.query('SELECT 1');
    res.json({ status: 'OK', database: 'Conectado com sucesso!' });
  } catch (err) {
    res.status(500).json({ status: 'ERRO', detalhe: err.message });
  }
});

// Endpoint para listar produtos
app.get('/api/produtos', async (req, res) => {
  try {
    const { rows } = await pool.query('SELECT * FROM produtos');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ erro: err.message });
  }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`API servindo na porta ${PORT}`);
});
```

1 . backend/Dockerfile:

Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

Passo 3: Código-Fonte da Camada Frontend (Web + Nginx)

1. **`frontend/index.html`:**

HTML

```html
<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <title>Painel Multicontainer Docker Compose</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px; background-color: #f4f4
        h1 { color: #333; }
        .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0
        .status-ok { color: green; font-weight: bold; }
    </style>
</head>
<body>
    <h1>Status da Aplicação Multicontainer</h1>
    <div class="card">
        <h2>Conexão com a API Backend e Banco:</h2>
        <p id="status">Carregando status...</p>
    </div>

    <script>
        fetch('/api/health')
            .then(res => res.json())
            .then(data => {
                document.getElementById('status').innerText = data.database;
                document.getElementById('status').className = 'status-ok';
            })
            .catch(err => {
                document.getElementById('status').innerText = 'Erro ao conectar na A
            });
    </script>
</body>
</html>
```

2. `frontend/nginx.conf`:

```nginx
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # Proxy reverso repassando requisições /api/ para o container da API
    location /api/ {
        proxy_pass http://backend:3000/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

3. `frontend/Dockerfile`:

```dockerfile
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Passo 4: Variáveis de Ambiente e Inicialização do Banco (`.env`)

Crie o arquivo **`.env`** na raiz do projeto:

```
Plaintext

POSTGRES_USER=app_admin
POSTGRES_PASSWORD=senha_mestra_super_segura
POSTGRES_DB=catalogo_db
PORTA_WEB=80
```

Passo 5: A Orquestração Declarativa (`docker-compose.yaml`)

Crie o arquivo docker-**`compose.yaml`** na raiz do projeto:

```yaml
YAML

services:
  # 1. Camada de Banco de Dados
  db:
    image: postgres:15-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - dados_pg:/var/lib/postgresql/data
    networks:
      - backend-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 5
```

```
  # 2. Camada de API / Backend
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    restart: unless-stopped
    environment:
      DB_HOST: db
      DB_USER: ${POSTGRES_USER}
      DB_PASSWORD: ${POSTGRES_PASSWORD}
      DB_NAME: ${POSTGRES_DB}
      PORT: 3000
    depends_on:
      db:
        condition: service_healthy
    networks:
      - backend-net
      - frontend-net
```

```
  # 3. Camada de Apresentação / Frontend
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    restart: unless-stopped
    ports:
      - "${PORTA_WEB}:80"
    depends_on:
      - backend
    networks:
      - frontend-net

# Definição de Redes Seguras
networks:
  frontend-net:
    driver: bridge
  backend-net:
    driver: bridge

# Definição de Volumes Persistentes
volumes:
  dados_pg:
    driver: local
```

Passo 6: O Desafio — Inicialização com Comando Único

Atendendo aos requisitos do desafio técnico, execute a inicialização completa do ambiente integrado através de **um único comando no terminal**:

Bash

```
docker compose up -d
```

Plaintext

```
[+] Running 5/5
 ✓ Network meu-projeto_frontend-net  Created
 ✓ Network meu-projeto_backend-net   Created
 ✓ Volume "meu-projeto_dados_pg"     Created
 ✓ Container meu-projeto-db-1        Healthy
 ✓ Container meu-projeto-backend-1   Started
 ✓ Container meu-projeto-frontend-1  Started
```

Passo 7: Validação e Diagnóstico de Problemas

1. **Verificar o Status e Saúde dos Containers:**

Bash

```
docker compose ps
```

Validar a Aplicação no Navegador ou Terminal: Acesse `http://localhost` ou execute:

```bash
curl http://localhost/api/health
```

Saída Esperada:

```json
{"status":"OK","database":"Conectado com sucesso!"}
```

Consultar Logs Agregados e Específicos:

```bash
# Ver os logs de todos os containers simultaneamente
docker compose logs

# Ver e acompanhar os logs do Banco de Dados
docker compose logs -f db
```

Simular Troubleshooting de Conectividade: Para inspecionar como o `frontend` enxerga a API internamente via DNS:

```bash
docker compose exec frontend ping -c 2 backend
```

Encerrar Limpando o Ambiente Completo: Quando o laboratório estiver concluído, interrompa os serviços preservando os dados do volume:

Bash

```
docker compose down
```

GUIA RÁPIDO DE COMANDOS DOCKER COMPOSE

Comando	Descrição e Propósito
`docker compose up -d`	Sobe todo o ambiente compilando imagens e criando redes/volumes em background.
`docker compose up -d --build`	Força a recompilação das imagens locais antes de iniciar os serviços.
`docker compose down`	Para e remove containers e redes criados pelo Compose.
`docker compose down -v`	Para, remove containers, redes e **apaga todos os volumes de dados**.
`docker compose ps`	Lista os serviços ativos do projeto e seus status de saúde (*healthcheck*).
`docker compose logs -f <servico>`	Exibe e acompanha os logs em tempo real de um serviço específico.
`docker compose exec <servico> <comando>`	Executa um comando dentro de um container ativo do serviço.
`docker compose config`	Valida a sintaxe do arquivo `compose.yaml` e exibe a configuração final resolvida.
`docker compose restart <servico>`	Reinicia o container de um serviço sem derrubar o restante da pilha.

REFERÊNCIAS

Livros e Obras de Referência

- **COMMUNITY, Docker.** *Docker in Action*. 2. ed. Manning Publications, 2019.
- **FRAYSER, Sean.** *Docker Deep Dive: Zero to Hero*. Independently published, 2023.
- **GARTNER, P.; SHARMA, S.** *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020.
- **KANE, Sean P.; MATTHIAS, Karl.** *Docker: Up & Running: Shipping Reliable Containers in Production*. 3. ed. Sebastopol: O'Reilly Media, 2023.
- **MOUAT, Adrian.** *Using Docker: Developing and Deploying Software with Containers*. Sebastopol: O'Reilly Media, 2015.
- **PULTORAK, David.** *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.
- **TURNBULL, James.** *The Docker Book: Containerization is the new virtualization*. OpenSource Press, 2014.

Recursos Web e Documentação Oficial

- **DOCKER.** *Docker Documentation*. Disponível em: https://docs.docker.com/. Acesso em: 07 out. 2026.
- **DOCKER HUB.** *Official Repositories and Image Registry*. Disponível em: https://hub.docker.com/. Acesso em: 07 out. 2026.
- **OPEN CONTAINER INITIATIVE (OCI).** *OCI Image Specification & Runtime Specification*. Disponível em: https://opencontainers.org/. Acesso em: 07 out. 2026.
- **SPRING.** *Spring Boot Reference Documentation: Efficient Container Images*. Spring / Broadcom. Disponível em: https://docs.spring.io/spring-boot/docs/current/reference/html/container-images.html. Acesso em: 07 out. 2026.
- **SWAGGER / SMARTBEAR.** *OpenAPI Specification*. Disponível em: https://swagger.io/specification/. Acesso em: 07 out. 2026.